

LAI

Legal Document Intelligence System

Architecture · Market Analysis · Design Tradeoffs · Future Evolution

GenAI Course Project · Group 21

The Contract Review Problem

Why legal document analysis is a hard AI problem



Volume

Enterprise legal teams review thousands of contracts per year. Manual review costs \$200–\$1,000 per contract-hour.



Complexity

A 100-page contract contains dozens of interdependent clauses with defined terms that cascade through the document.



Risk

Missing one unfavourable indemnity or uncapped liability clause can result in catastrophic financial exposure.



Expertise

Effective review requires legal domain knowledge: jurisdiction, clause precedents, and market-standard positions.

The core AI challenge: extract structured meaning from unstructured legal language, reason across interdependent clauses, and produce reliable risk assessments grounded in market precedent.

Industry Architecture Patterns

The four generations of Legal AI system design

01 Prompt-only

Contract text → LLM prompt → output.
No retrieval, no tools, no structure enforcement.

Example: early Spellbook, GPT wrappers.

Limitation: hallucinates facts, loses long docs.

02 Long-context

Full document in context window → structured extraction → analysis.

Example: LAI, Harvey for single contracts.

Strength: preserves cross-clause context.

Limitation: cost, no external knowledge.

03 RAG systems

Clause extracted → embedded → similar clauses retrieved from corpus → LLM reasons on evidence.

Example: Luminance, Evisort (repo search), Harvey (firm precedents).

Strength: grounded, updatable, explainable.

Limitation: retrieval quality = corpus quality.

04 Agentic systems

LLM orchestrates multi-step loop: extract → cross-reference → retrieve → reason → redline → memo.

Example: Harvey (2024+), Robin AI negotiation agent.

Strength: holistic cross-clause reasoning.

Limitation: slow, expensive, hard to audit.

Deep Dive: RAG vs Long-Context

When each architecture is the right choice for legal document analysis

Long-Context (LAI's approach)

Advantages

- ✓ Full cross-clause context — defined terms, schedules, cross-references all visible
- ✓ No chunking errors — clause meaning preserved in full document context
- ✓ Simpler architecture — no vector index, no retrieval pipeline
- ✓ Works on first document — no corpus needed to get started

Limitations

- ✗ Higher token cost per analysis
- ✗ No external knowledge — market norms frozen at training time
- ✗ Cannot scale to multi-contract portfolio queries
- ✗ Context window is still finite (~1M tokens practical limit)

RAG (next evolution)

Advantages

- ✓ Empirically grounded — compares against real executed contracts
- ✓ Updatable corpus — market knowledge improves without retraining
- ✓ Scales to portfolio queries — retrieve across thousands of contracts
- ✓ Explainable — citations point to specific retrieved precedents

Limitations

- ✗ Corpus quality is the moat — poor corpus = poor comparisons
- ✗ Retrieval can miss — a clause unlike any in the corpus is poorly served
- ✗ More complex architecture — vector index, embeddings, retrieval pipeline
- ✗ Legal-domain embeddings needed — general embeddings underperform

Deep Dive: Contract Comparison Approaches

Four architectures for market-standard clause comparison

01 Static templates

Hand-written description of acceptable clause language. Exact string match or keyword check.

Quality: Low — brittle to paraphrasing, reflects author opinion only.

Used by: Early Lawgeex, basic CLMs.

Superseded

02 Baseline DB lookup

Structured market-standard positions per clause type stored in DB. LLM compares extracted clause against retrieved baseline text.

Quality: Medium — grounded but editorial. As good as whoever wrote the baselines.

Used by: LAI, Lawgeex playbook engine.

LAI today

03 RAG benchmark

Real clauses from executed contracts (CUAD, proprietary corpus) embedded and indexed. Retrieve similar clauses at query time. LLM reasons against real examples.

Quality: High — empirically grounded, explainable, updatable.

Used by: Luminance, Evisort, Harvey.

LAI future

04 Knowledge graph

Clause obligations, parties, risk concepts as graph nodes. Edges encode relationships and jurisdiction-specific rules. Symbolic + neural reasoning.

Quality: Highest potential — systemic risk visible across clause interactions.

Used by: Research stage; no production deployment confirmed.

Frontier

LAI System Architecture

End-to-end pipeline: ingestion → extraction → analysis → output

Upload

PDF / DOCX

Docling Parse

Markdown + JSON

Gemini Extract

Structured JSON schema

Hybrid Score

60% LLM + 40% rules

Summary Gen

1-page plain English

Output

Report + Chat + Compare

Document Processing

- ▶ IBM Docling: layout-aware parsing preserves clause hierarchy, lists, tables, section numbering
- ▶ Output: structured Markdown + JSON document object model
- ▶ Advantage over naive extraction: retains cross-reference context
- ▶ Parsed content persisted to PostgreSQL before analysis begins

Clause Extraction + Risk

- ▶ Gemini 2.0 Flash with structured JSON schema output
- ▶ 8 named clause types: indemnity, liability, governing law, termination, confidentiality, IP, payment, and more
- ▶ Hybrid risk scoring: 60% Gemini semantic + 40% regex rule-based
- ▶ Market comparison: baseline text retrieved from PostgreSQL by clause type

Output Features

- ▶ Executive summary: plain-English 1-page report generated from extracted clauses
- ▶ Contract chat: full document in context window, last 5 messages included
- ▶ Clause comparison: direct DB query by clause type across multiple contracts
- ▶ Risk taxonomy: financial, legal, operational, reputational categories

Architectural Decisions & Tradeoffs

Every design choice optimises for something — and costs something else

Decision	Benefit	Tradeoff
IBM Docling for parsing	Preserves clause hierarchy, section numbering, tables — maintains cross-reference context for LLM	Dependency on Docling quality; heavier pipeline than naive pdfplumber extraction
Long-context window	Full document visible at once — no chunking errors, no lost cross-clause dependencies	Higher token cost per analysis; no external knowledge access; slower per contract
Gemini structured output	Reliable JSON schema enforcement — downstream processing and DB storage simplified significantly	Tied to Gemini API; structured output has token overhead; schema must match clause reality
Hybrid risk scoring 60/40	Deterministic rule-based component prevents LLM-only variance; semantic component handles nuance	Weight ratio (60/40) is heuristic, not empirically calibrated; rules require ongoing maintenance
Baseline DB for market compare	Inspectable, auditable, consistent — baseline text is human-readable and lawyer-editable	Quality bounded by what the baselines say; no real market data; editorial not empirical
Full-doc contract chat	Complete document awareness — answers cross-referencing multiple sections are accurate	Higher token usage and latency than RAG-based chat; no persistent memory across sessions
Direct DB clause compare	Zero retrieval uncertainty for multi-contract clause comparison — exact match by type	Cannot discover similar clauses in unknown contracts; requires prior extraction and classification

Benefits in green · Tradeoffs in red · These are honest assessments, not marketing claims

LAI vs Competitors

Honest feature-by-feature positioning

Clause extraction

LAI strength

LAI: Gemini structured JSON — 8 named types, reliable schema output

Market: Kira: 1000+ classifier types (breadth). Harvey: zero-shot any type (flexibility).

Risk scoring

LAI strength

LAI: Hybrid 60/40 rule+LLM with financial/legal/operational/reputational taxonomy

Market: Most competitors: numeric score only with no taxonomy breakdown.

Market comparison

Competitor ahead

LAI: Baseline DB lookup — consistent but editorial. Gemini judges deviation.

Market: Luminance/Evisort: vector similarity against real corpus. Harvey: firm precedent RAG.

Document processing

LAI strength

LAI: IBM Docling preserves hierarchy, structure, tables — better than naive extraction

Market: Most CLMs: basic text extraction. Harvey/Robin: similar pipeline quality.

Contract chat

Comparable

LAI: Full long-context, last 5 messages — accurate cross-clause answers

Market: Harvey: RAG-based chat (cheaper but can miss context). Spellbook: Word in-context.

Agents & memory

Competitor ahead

LAI: No agents. Session-only memory (last 5 messages). No persistent state.

Market: Harvey/Robin AI: multi-step agents, autonomous review.

Current Strengths of LAI

What the architecture does well and why



Structure-aware ingestion

IBM Docling preserves clause hierarchy, section numbering, and tables. The document is a structured object, not a text dump. Cross-references remain resolvable.



Structured clause extraction

Gemini with schema enforcement produces typed clause objects with metadata. Downstream processing, DB storage, and comparison all operate on structured data.



Hybrid risk scoring

60% LLM semantic + 40% rule-based. The rule component prevents hallucinated risk assessments; the LLM component handles nuanced language the rules miss.



Risk taxonomy

Financial, legal, operational, reputational categories. Most competitors produce a single numeric score. Taxonomy-based scoring is rare and analytically useful.



Long-context contract chat

Full document in context window. Answers to cross-referencing questions (e.g., 'does the indemnity conflict with clause 12?') are accurate because the model sees everything.



Direct clause comparison

Clauses extracted and typed in DB. Multi-contract comparison queries directly by type — no retrieval uncertainty. Fast, deterministic, and auditable.

Future Evolution Roadmap

Prioritised architectural improvements and the value each adds

1

RAG for market comparison

Replace editorial baselines with CUAD-backed retrieval. Comparison becomes empirically grounded — real executed contracts, not hand-written descriptions.

Impact: High

Effort: Medium

2

RAG for contract chat

Replace full-document context with clause-level retrieval for chat. Reduces token cost by 10–100x while retaining answer quality for most queries.

Impact: High

Effort: Medium

3

Multi-contract retrieval

Vector-index all extracted clauses. Enable queries like 'find all contracts with uncapped indemnities' across the full portfolio — impossible with current architecture.

Impact: High

Effort: Medium

4

Agentic contract reviewer

LangGraph orchestrator: extract → cross-reference → retrieve precedents → score → redline → memo. Enables cross-clause risk detection invisible to clause-by-clause review.

Impact: V. High

Effort: High

5

Persistent memory

Store user preferences, prior review decisions, and correction history. Active learning loop — human corrections improve extraction quality over time.

Impact: Medium

Effort: Medium

6

LangGraph orchestration

Replace linear pipeline with graph-based workflow. Enables parallel analysis, conditional branching, retry logic, and clean separation of agent concerns.

Impact: Medium

Effort: High

Key Takeaways

Lessons from market research and architecture analysis

01

Long-context and RAG solve different problems

Long-context preserves within-document clause relationships. RAG grounds comparison in external market data. A mature system needs both — use each where it wins.

02

Hybrid scoring is architecturally sound

Pure LLM risk scoring is inconsistent. Pure rules are brittle. The 60/40 hybrid approach combines semantic understanding with deterministic reliability — a real design improvement.

03

Structure is the foundation

Docling's structure-aware parsing is the decision the rest of the pipeline depends on. Clause hierarchy, defined-term resolution, and cross-references only work if the document model preserves them.

04

The market comparison gap is solvable

Replacing editorial baselines with CUAD-backed RAG retrieval is the highest-priority evolution. The infrastructure (pgvector in existing PostgreSQL) requires no new dependencies.

05

Agents require long-context reasoning first

Agentic review only works when the underlying extraction and risk scoring are reliable. LAI's current pipeline is the correct foundation — agent orchestration is the next layer, not a replacement.

06

Explainability is the unsolved frontier

No competitor can reliably cite which clauses justify a risk score. Faithful attribution — every claim traceable to a source span — is the most valuable open problem in production Legal AI.